

Creating and managing blocks

Manual

20.00.00.00

11/2024

NOTICE
This document was downloaded from the following site: https://docs.tia.siemens.cloud Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided. Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

 DANGER
indicates that death or severe personal injury will result if proper precautions are not taken.
 WARNING
indicates that death or severe personal injury may result if proper precautions are not taken.
 CAUTION
indicates that minor personal injury can result if proper precautions are not taken.
NOTICE
indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:



WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

11/2024 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1	Creating and managing blocks	6
1.1	Creating blocks	6
1.1.1	Block folder	6
1.1.2	Creating organization blocks	7
1.1.3	Creating functions and function blocks	8
1.1.4	Creating data blocks	9
1.1.5	Using blocks from libraries	11
1.1.6	Copying or moving blocks	12
1.1.6.1	Basics of copying blocks	12
1.1.6.2	Copying or moving blocks	16
1.1.7	Entering a block title	16
1.1.8	Entering a block comment	17
1.2	Specifying block properties	18
1.2.1	Basics of block properties	18
1.2.2	Overview of block properties	18
1.2.3	Block time stamps	25
1.2.4	Displaying and editing block properties	27
1.3	Managing blocks	28
1.3.1	Opening blocks	28
1.3.2	Saving blocks	30
1.3.3	Closing blocks	30
1.3.4	Renaming blocks	31
1.3.5	Deleting blocks offline	31
1.3.6	Deleting blocks online (S7-300, S7-400)	32
1.3.7	Deleting CPU data blocks	33
1.4	Using external source files for STL and SCL	33
1.4.1	Basics of using external source files	33
1.4.2	Rules for programming external source files	35
1.4.3	Saving blocks as external source files	36
1.4.4	Inserting external source files	38
1.4.5	Opening and editing external source files	39
1.4.6	Generating blocks from external source files	40
1.5	Exporting and importing blocks in SIMATIC SD format (S7-1200, S7-1500)	40
1.5.1	Exporting and importing blocks in SIMATIC SD format (S7-1200, S7-1500)	41
1.5.2	Representation of LAD program code in SIMATIC SD format (S7-1200, S7-1500)	42

Creating and managing blocks

This section contains information on the following topics:

- Creating blocks
- Specifying block properties
- Managing blocks
- Using external source files for STL and SCL
- Exporting and importing blocks in SIMATIC SD format (S7-1200, S7-1500)

1.1 Creating blocks

1.1.1 Block folder

Function

You can find a "Program blocks" folder in the project tree, in which you can create and manage the following blocks:

- Organization blocks (OB)
- Function blocks (FB)
- Functions (FCs)
- Data blocks (DB)

A "System blocks" subfolder containing another subfolder, "Program resources", is also created in the "Program blocks" folder the first time you drag an instruction to your program which is an internal system function block. The instance data block of the internal system function block is also pasted to the "Program resources" folder. You can copy, rename or delete such instance data blocks in the "Program resources" folder. The "Program resources" folder is deleted together with the "System blocks" folder if contains no more blocks.

A program cycle OB is automatically generated for each device and inserted in the "Program blocks" folder.

Note

Moving blocks from the "Program resources" folder or to the "Program resources" folder

You can move blocks between the "Program resources" folder and other folders. When moving blocks from the "Program resources" folder or to the "Program

resources" folder, note the following:

- Moving blocks from the "Program resources" folder into another folder: When the next time the user program is compiled, the shifted instance data blocks are moved back into the "Program resources" folder.
- Moving by any blocks to the "Program blocks" folder: Blocks in the "Program resources" folder that are not required to run the user program are removed during the next compilation. This may cause your moved block to be deleted.

Grouping blocks

Within the "Program blocks" folder, you can create subfolders (groups) to structure your blocks. These subfolders are not runtime-related, the blocks contained there are, however. When you load your program to a device and then load it to your project again later, these subfolders are retained.

See also

Creating functions and function blocks

Creating data blocks

Creating organization blocks

Using blocks from libraries

1.1.2 Creating organization blocks

Requirement

The "Program blocks" folder in the project tree is open.

Procedure

To create an organization block, follow these steps:

1. Double-click the "Add new block" command.
The "Add new block" dialog box opens.
2. Click the "Organization block (OB)" button.
3. Select the type of new organization block.
4. Enter a name for the new organization block. The name can contain up to 125 characters when no namespace has been defined for the block.
5. Optional within software units: Enter a namespace for the new organization block or use the namespace preset of the software unit.

Note

Namespaces in software units

You can find information on namespaces, in particular on the naming conventions according to IEC 61131-3, here:

Categorizing program elements in namespaces

6. Enter the properties of the new organization block.
7. To enter additional properties for the new organization block, click "Additional information".
An area with further input fields is displayed.
8. Enter all the properties you require.
9. Activate the "Add new and open" check box if the organization block does not open as soon as it is created.
10. Confirm your entries with "OK".

Result

The new organization block is created. You can find the organization block in the project tree in the "Program blocks" folder. You can assign additional parameters to some organization blocks in the inspector window or device view after they have been created. The organization block description will state whether the newly created organization block has additional parameters.

See also

Organization blocks (OB)
Block folder
Creating functions and function blocks
Creating data blocks
Using blocks from libraries
Entering a block title
Entering a block comment

1.1.3 Creating functions and function blocks

Requirement

The "Program blocks" folder in the project tree is open.

Procedure

To create a function (FC) or a function block (FB), follow these steps:

1. Double-click the "Add new block" command.
The "Add new block" dialog box opens.
2. Click the "Function block (FB)" or "Function (FC)" button.

3. Enter a name for the new block. The name can contain up to 125 characters when no namespace has been defined for the block.
4. Optional within software units: Enter a namespace for the new block or use the namespace preset of the software unit.

Note

Namespaces in software units

You can find information on namespaces, in particular on the naming conventions according to IEC 61131-3, here:

Categorizing program elements in namespaces

5. Enter the properties of the new block.
6. To enter additional properties for the new block, click "Additional information".
An area with further input fields is displayed.
7. Enter all the properties you require.
8. Activate the "Add new and open" check box if the block does not open as soon as it is created.
9. Confirm your entries with "OK".

Result

The new block is created. You can find the block in the project tree in the "Program blocks" folder.

See also

Function blocks (FB)
Functions (FCs)
Basics of block access
Block folder
Creating organization blocks
Creating data blocks
Using blocks from libraries
Entering a block title
Entering a block comment

1.1.4 Creating data blocks

Requirement

The "Program blocks" folder in the project tree is open.

Procedure

To create a data block, follow these steps:

1. Double-click the "Add new block" command.

The "Add new block" dialog box opens.

2. Click the "Data block (DB)" button.
3. Select the type of the data block. You have the following options available to you:
 - To create a global data block, select the list entry "Global DB".
 - To create an ARRAY data block, select the "ARRAY DB" entry in the list.
 - To create an instance data block, select the function block to which you want to assign the instance data block from the list. The list contains only the function blocks that were previously created for the CPU.
 - To create a data block based on a PLC data type, select the PLC data type from the list. The list contains only the PLC data types that were previously created for the CPU.
 - To create a data block based on a system data type, select the system data type from the list. The list contains only those system data types that have already been inserted to program blocks in the CPU.
4. Enter a name for the data block. The name can contain up to 125 characters when no namespace has been defined for the data block.
5. Optional within software units: Enter a namespace for the new data block or use the namespace preset of the software unit.

Note

Namespaces in software units

You can find information on namespaces, in particular on the naming conventions according to IEC 61131-3, here:

Categorizing program elements in namespaces

6. Enter the properties of the new data block.
7. If you have selected an ARRAY DB as the data block type, enter the ARRAY data type and the high limit for the ARRAY.

You can change the high limit for the ARRAY at any time in the property window of the created block. The ARRAY data type cannot be changed subsequently.
8. If you have selected a block that contains monitoring as the "Type", assign a ProDiag function block to the monitoring functions.
9. To enter additional properties of the new data block, click "Additional information".

An area with further text boxes is displayed.
10. Enter all the properties you require.

11. Activate the "Add new and open" check box if the block does not open as soon as it is created.
12. Confirm your entry with "OK".

Result

The new data block is created. You can find the data block in the project tree in the "Program blocks" folder.

Note

Retentivity of ARRAY data blocks

ARRAY data blocks and their components cannot be set retentively.

See also

Basic principles for programming of data blocks
Global data blocks (DB)
Instance data blocks
System data types
Using blocks from libraries

1.1.5 Using blocks from libraries

You can save blocks in the project library or in a global library, so that you can use them more than once within a user program. You can insert the blocks as master copies or as types.

See also: Library basics

Requirement

- The "Libraries" task card is displayed.
- No write protection is set for global libraries.

Adding blocks as master copies to the project library or to a global library

To add new blocks as master copies to the project library or to a global library, follow these steps:

1. Maximize the project library or the global library.
2. Use drag-and-drop to move the block you wish to add to the library to the "Master copies" folder or any one of the "Master copies" subfolders in the project library or a global library. Do not release the left mouse button until a small plus sign appears underneath the mouse pointer.

Or:

1. Copy the block you want to add as master copy.

2. Maximize the project library or the global library.
3. Right-click the "Master copies" folder or any of its subfolders.
4. Select "Paste" in the shortcut menu.

Adding blocks as types to the project library or to a global library

To add new blocks as types to the project library or to a global library, follow these steps:

1. Maximize the project library.
2. Drag-and-drop the block you want to add as a type into the "Types" folder or any subfolder of "Types" in the project library. Do not release the left mouse button until a small plus sign appears underneath the mouse pointer.
3. If necessary, drag the block from the "Types" folder of the project library into the "Types" folder of a global library. Or you can copy the block in the project library, and add it to a global library.

Or:

1. Copy the block you want to add as type.
2. Maximize the project library.
3. Right-click the "Types" folder or any of its subfolders.
4. Select "Paste" in the shortcut menu.
5. If necessary, drag the block from the "Types" folder of the project library into the "Types" folder of a global library. Or you can copy the block in the project library, and add it to a global library.

Using blocks of the project library or a global library

To use a block from the project library or a global library in your project, follow these steps:

1. Maximize the project library or the global library so that you can see the block you wish to use.
2. Use a drag-and-drop operation to move the block to the CPU block folder. If the selected insertion points is not allowed, the mouse pointer will appear as a circle with a slash.

Note

If you derive an instance from a type in a global library, the type is also inserted into the project library. The instance is then only linked to the type in the project library.

1.1.6 Copying or moving blocks

1.1.6.1 Basics of copying blocks

Function

You can also create new blocks by copying existing blocks and pasting the copy. Note the following principles when copying to CPUs of the same device family:

- You can copy organization blocks (OBs), functions (FCs), function blocks (FBs), and global data blocks (DBs) without restriction.
- You can copy instance data blocks only for the same function block, since the assignment to the function block cannot be changed afterwards. However, the assignment is canceled if you copy the instance data block to a different CPU. If a function block with the same name exists there, the instance data block will be assigned to this function block. If you copy the instance data block together with the function block into the other CPU, the instance data block is assigned to the copy of the function block.

The various device families sometimes support different blocks, especially in the case of organization blocks. However, function blocks and functions can also be programmed on the various devices with different access types. Therefore, not all blocks are supported on all devices. Note the following principles when copying to a different device family:

- Copying to an S7-1200 CPU:
 - Organization blocks with "Optimized" access type can be copied to an S7-1200. If the copied OB type is supported by the S7-1200 CPU, the copied OB retains the properties of its event. You must, however, compile it again.
 - Although organization blocks with the "Standard" access type can be copied to an S7-1200, they are not supported by the CPU.
 - Function blocks (FBs), functions (FCs) and global data blocks (DBs) with "Optimized" access type can be copied to an S7-1200. However, they must be recompiled after this.
 - Instance data blocks: If there is a function block in the target CPU with the name that was assigned to the instance data block in the source CPU, the instance data block is assigned to the function block in the target CPU. If you copy the instance data block together with the function block to which it was assigned in the source CPU into the target CPU, the instance data block is assigned to the copy of the function block.
- Copying to an S7-1500 CPU:
 - Organization blocks with "Optimized" access type can be copied to an S7-1500. If the copied OB type is supported by the S7-1500 CPU, the copied OB retains the properties of its event. You must, however, compile it again. OB types that are not supported receive a no parking symbol.
 - Organization blocks with "Standard" access type can be copied to an S7-1500. If the OB derives from an S7-300/400 CPU, it receives the standard event of the corresponding OB type. If the OB derives from an S7-1200/1500 CPU, it receives the properties of its event. However, it must be compiled again.

- Function blocks (FBs), functions (FCs) and global data blocks (DBs) with "Optimized" access type can be copied to an S7-1500. However, they must be recompiled after this.
- Instance data blocks: If there is a function block in the target CPU with the name that was assigned to the instance data block in the source CPU, the instance data block is assigned to the function block in the target CPU. If you copy the instance data block together with the function block to which it was assigned in the source CPU into the target CPU, the instance data block is assigned to the copy of the function block.
- Copying to S7-300/400 CPUs:
 - Organization blocks can be copied as required between S7-300 and S7-400.
 - Although organization blocks from S7-1200/1500 CPUs can be copied to S7-300/400 CPUs, they are not supported by the target CPU.
 - Function blocks (FBs), functions (FCs) and global data blocks (DBs) can be copied as required between S7-300 and S7-400.
 - Although function blocks (FBs), functions (FCs) and global data blocks (DBs) can be copied from S7-1200/1500 CPUs to S7-300/400 CPUs, they are not supported by the target CPU.
 - Instance data blocks: If there is a function block in the target CPU with the name that was assigned to the instance data block in the source CPU, the instance data block is assigned to the function block in the target CPU. If you copy the instance data block together with the function block to which it was assigned in the source CPU into the target CPU, the instance data block is assigned to the copy of the function block.
 - You may be able to copy function blocks (FBs) and functions (FCs) that are programmed with LAD or FBD and contain SCL networks to S7-300/400 CPUs, but they are not supported by the target CPU.

In the project tree, blocks that are not supported are indicated by the no parking symbol. Blocks with a no parking symbol cannot be edited, but only used again as copy source.

Note

When blocks are copied between different device families, it is possible that the copied block needs to be recompiled. This applies also to the copying of blocks between CPUs and software controllers. If the block has know-how protection, re-compilation is only possible with the correct password.

Copying data

With paste, all the block data is copied and forwarded to the copy. This data includes:

- Block interface tags

- All networks
- Comments in all existing compilations
- Messages defined in the block
- The entire program code of the copied block including the call instructions contained in the block.

However, the following objects are not copied:

- Called blocks and associated instance data blocks
- Global tags that are referenced in the copied block
- The PLC data types used in the copied block for the declaration of the block parameters and local tags
- The technology objects used in the copied block

Avoiding name conflicts during pasting

When pasting copied blocks with identical names to already existing blocks, the following mechanisms are used to avoid name conflicts:

- Pasting the copied block into the same CPU:

The copy of the block gets a name that is extended by a number. For example, if block "A" is copied, a possible name for the copy is "A_1". A consecutive number range is not used for the number. Rather the smallest free number is always used. The copy of block "A" can also get the name "A_25", if no lower number is available.

- Pasting the copied block into another CPU:

A dialog box opens in which you can select whether the block with the same name will be replaced or the copied block will be pasted with a duplicate designation (Name_Number).

In software units, namespaces help avoid name conflicts. For more information on how namespaces are handled when copying blocks, see "Basics for copying or moving program elements".

Note

Number conflicts

Number conflicts may occur, if the pasted block has the same block number as an existing block. The block number is not automatically changed during pasting. This double number may have an effect on block calls. When you copy blocks you should therefore check the block number carefully and correct duplicate block numbers manually or using the block properties. Duplicate block numbers lead to a compilation error.

See also

1.1.6.2 Copying or moving blocks

Requirement

The "Program blocks" folder in the project tree is open.

Copying blocks

To copy a block, follow these steps:

1. Right-click the block that you want to copy.
2. Select the "Copy" command in the shortcut menu.
3. In the project tree, right-click the program block folder into which you want to insert the copied block.
4. Select the "Paste" command in the shortcut menu.
 - If you paste the block into the same CPU, the copy is pasted with the name extension "_<consecutive number>".
 - If you paste the block into a different CPU where a block of the same name already exists, the "Paste" dialog box opens. Select the desired option and confirm your selection with "OK".

Alternatively, you can also copy the block via drag-and-drop while holding down the <Ctrl> key.

Moving blocks

To move a block, follow these steps:

1. Select the block that you want to move.
2. Drag-and-drop the block to the new position.

See also

Basics of copying blocks

1.1.7 Entering a block ft:title

The block title is the title of the block. It is not the same thing as the block name, which was assigned when the block was created. The length of the block title is restricted to one row. You can enter the block title for open and closed blocks.

Requirement

A code block is available.

Enter block title for open block

To insert the block title in an open block, follow these steps:

1. Click on the title bar of the block in the program editor.
2. Enter the block title.

Enter block title for closed blocks

To insert the block title in a closed block, follow these steps:

1. Right-click the block in the project tree.
2. Select the "Properties" command in the shortcut menu.

The dialog with the properties of the block opens.

3. Select the entry "Information" in the area navigation.
4. Enter the block title in the "Title" input field.
5. Confirm your entry with "OK".

See also

Creating organization blocks

Creating functions and function blocks

Entering a block comment

1.1.8 Entering a block comment

You can use the block comment to document the entire code block. For example, you can indicate the purpose of the block or draw attention to special characteristics. You can enter the block comment for open and closed blocks.

Note

The block comment should not exceed 32,767 Unicode characters.

Requirement

A code block is available.

Enter block comment for open blocks

To insert a block comment in an open block, proceed as follows:

1. Click the small arrow in front of the block title.
The right arrow becomes a down arrow, and the comment area is displayed.
2. Click "Comment" in the comment area.
The "Comment" text passage is selected.
3. Enter the block comment.

Enter block comments for closed blocks

To insert the block comment in a closed block, follow these steps:

1. Right-click the block in the project tree.
2. Select the "Properties" command in the shortcut menu.
The dialog with the properties of the block opens.
3. Select the entry "Information" in the area navigation.
4. Enter the block comment in the "Comment" input field.
5. Confirm your entry with "OK".

See also

Creating organization blocks

Creating functions and function blocks

Entering a block title

1.2 Specifying block properties

1.2.1 Basics of block properties

Block properties

Each block has certain properties, which you can display and edit. These properties are used amongst other things to:

- Identify the block
- Display the memory requirements and the compilation status of the block
- Display the time stamp
- Display the reference information
- Specify the access protection

See also

Overview of block properties

Block time stamps

Displaying and editing block properties

Setting the mnemonics

1.2.2 Overview of block properties

Overview

The properties of the blocks are block and CPU-specific. Not all properties are therefore available for all blocks or in all CPU families. The following table gives an overview of block properties:

Group	Property	Description
Properties	Name	Unique block name within the station
	Constant name	Name of the constant pasted for the OB in the PLC tag table
	Type	Block type (cannot be changed)
	Number	Block number
	Event class	Event class of an OB (cannot be changed)
	Language	Programming language of the block
	Language in networks	Language used to program conditions in GRAPH blocks.
	Version	Version of the GRAPH block
	Process image partition number	Display of the process image partitions that are assigned to the organization block (cannot be changed)
	ARRAY data type	Data type of an ARRAY data block (cannot be changed)
	ARRAY limit	High limit of an ARRAY data block The "Move operations" section of the "Instructions" task card offers options for addressing of ARRAY data blocks.
Information	Title	Block title
	Comment	Block comment
	Version	Version number of the block
	Family	Block family name

Information	Author	Name of the author, company name, department name, or other names
	User-defined ID	ID created by the user
Time stamp	Block	Times of creation and time of change of the block (cannot be changed)
	Interface	Time of creation of the block interface (cannot be changed)
	Code	Code modification time (non-editable)
	Data	Data modification time (non-editable)
	Load-relevant	Time of the last load-relevant modification (cannot be changed)
Compilation	Status	Details of the last compilation run (cannot be changed)
	Lengths	Details of the block lengths (cannot be changed)
	Support for simulation and virtual CPUs	• Can be simulated with SIMATIC S7-PLCSIM Indicates whether the block can be simulated with SIMATIC S7-PLCSIM Advanced for compiling. This requires the "Support simulation during block compilation" option to be selected in the project properties. See also: Displaying properties of the project • Can be used in virtual S7-1500 CPUs Enables the block to be used on virtual S7-1500 CPUs. See also: Principle of operation of virtual CPUs
	Library conformance	Shows whether or not the block can be added as type to a library.

Protection	Protection	Setting up know-how protection and copy protection for the block See also: Protecting blocks
Attributes	IEC check	The compatibility of the operands in comparison operations and arithmetic operations are tested according to IEC 61131. You have to explicitly convert non-compatible operands. See also: Overview of data type conversion
	Local error handling within block	Troubleshooting inside the block with the GET_ERROR or GET_ERR_ID instruction (cannot be changed). See also: Handling program execution errors
	Optimized block access	The tag declaration for blocks with optimized access contains only the symbolic names of the data elements. The system automatically optimizes and manages the addresses. The CPU performance increases and access errors, for example, from SIMATIC HMI, are prevented. See also: Blocks with optimized access
	Capable of multi-instance	The block can be called with the "Multi-instance" call option. This attribute cannot be changed.
	Create extended status information	Allows you to monitor all tags in an SCL block. This option does, however, increase the program memory space required and the execution times.
	Check ARRAY limits	Checks whether array indexes are within the range declared for an ARRAY during the runtime of an SCL block. The block enable output ENO is set to "0" if an array index is outside of the permitted range.

Attributes	Set ENO automatically/Set ENO automatically for SCL blocks and SCL networks	Checks whether errors occur in the processing of certain instructions during the runtime of an SCL block. The block enable output ENO is set to "0" if a runtime error occurs.
	Create minimized DB	Creates instance data blocks for GRAPH blocks of the S7-300 and S7-400 in minimized format. This option reduces the GRAPH FB memory space required, however you will only receive limited program status information.
	Skip steps	If the transitions before and after a step become valid simultaneously in a GRAPH block, the step will not be activated and will be skipped.
	Acknowledge supervision alarms	Any supervision error which occurs during the operation of a GRAPH block must be acknowledged before the program can continue.
	Permanent processing of all interlocks in manual operation	Permanently monitors all interlock conditions in a GRAPH block in manual mode.
	Initial value acquisition (Can only be activated as of block version V4.0)	The signal state of operands and the results of comparisons from a step or a transition are stored in the static parameters "CRIT_LOC", "CRIT_LOC_ERR", "CRIT" and "CRIT_FLT" of the GRAPH block interface.
	Lock operating mode selection	Prevents a GRAPH block operating mode being selected.
	Name of the expansion block	Defines the extension block via which you can access the invisible parameters in the block interface of the GRAPH function block.
	Data block write-protected in the device	Indicates whether the data block is read-only in the target system, and cannot be overwritten while the program is running. The option is available only for data blocks.

Attributes	Only store in load memory	When activated, only management information (64 bytes) is stored for the block in the load memory. The program can therefore not directly access the tags of the block. The "Instructions" task card in the "Extended Instructions > Data block functions" palette provides access options. The option is available only for data blocks.
	Data block accessible from OPC UA	When the check box is selected the data block is accessible as complete object from OPC UA. The individual tags of the data block can then be individually released or locked for OPC UA. This option is only available for data blocks of the S7-1200/S7-1500.
	Data block accessible via web server	If the check box is selected, the data block can be accessed as a whole object via a web server. The individual tags of the data block can then be released or locked separately. The option is only available for data blocks.
	Start information	Here you specify the structure of the start information of the OB for S7-1500 CPUs: either like for S7-300 and S7-400 CPUs or optimized start information.
	Priority	Shows the priority set for organization blocks. The CPU family used and the type of the organization block determine whether you can change the priority.
	Parameter input via registers	Enables parameter input by means of registers in an STL block of S7-1500. This feature allows you to use the "Conditional call of blocks" (CC) and "Unconditional call of blocks" (UC) instructions in the block.
	Enable readback	Enables you to mark individual parameters of the block for readback. The "Tag readback" function is relevant if the block is used in a CFC chart.
	Block representation	Specifies how the block is represented in a CFC chart. You can find or more information in the help for CFC on the page "Display variants of the block symbol in

	the CFC chart".	
Time-of-day interrupt	Time-of-day interrupt	Parameters of the time-of-day interrupt OB: active (yes or no), execution, start date and time, local time or system time
Cyclic interrupt	Cyclic interrupt	Cycle time and phase offset of the cyclic interrupt OB
Triggers	Triggers	Display of start events of the hardware interrupt OB
Isochronous mode	Isochronous mode	Parameters of the isochronous mode interrupt OB: application cycles, automatic setting (yes or no), delay time. Also indicated is the PROFINET IO system or DP master system whose IO devices or DP slaves are assigned to the isochronous mode interrupt OB.
Download without reinitialization	Memory reserve	Defines the reserve in volatile memory that can be used for interface extensions. The number of currently available bytes is displayed in parentheses. This information is updated with each compilation.
	Activate download without reinitialization for retentive tags	Enables definition of a reserve in retentive memory.
	Reserve in retentive memory	Defines the reserve in retentive memory that can be used for interface extensions. The number of currently available bytes is displayed in parentheses. This information is updated with each compilation.

See also

Overview of organization blocks (S7-300, S7-400)
Events and OBs (S7-1200)
Events and OBs (S7-1500)
Basics of block properties
Block time stamps
Displaying and editing block properties
Basics of block access
General settings for the PLC programming

1.2.3 Block time stamps

Introduction

Blocks receive a number of different time stamps which show you when the block was created and when it was last changed. These time stamps are also used for automatic consistency checks before compilation.

Code block time stamps

The following time stamps are generated for code blocks (OBs, FBs, FCs):

- Block: Created on, Modified on
- Interface: Modified on
- Code/data: Modified on
- Load-relevant: Modified on

A time stamp conflict is displayed during compilation if the time stamp for the block calling is older than that of the interface for the block called.

Time stamps for code blocks are updated as follows:

- Block: The time stamp for the last block modification is always the same as the time stamp either of the interface or of the code depending on which area was modified last.
- Interface: The interface time stamp is updated each time the interface is modified. Even if you manually undo a change to the interface, for example change the name back, that is also a change which updates the time stamp. However, if you undo the change using the "Undo" function, the time stamp will be reset to the value it had before the undone change.
- Code/data: The code time stamp is updated each time the block code is changed. Even if you manually undo a change to the code, for example remove an instruction, that is also a change which will update the time stamp. However, if you undo the change using the "Undo" function, the time stamp will be reset to the value it had before the undone change.
- Load-relevant: The time stamp for "Load-relevant" is updated each time a code block is changed. These changes can affect the code, the data or the interface.

Global data block time stamps

The following time stamps are generated for global data blocks:

- Block: Created on, Modified on
- Interface: Modified on
- Data: Modified on
- Load-relevant: Modified on

Time stamp conflict is reported during compilation of a global data block based on a PLC data type if the time stamp of the global data block is older than the time stamp of the PLC data type used.

The time stamps for global data blocks are updated as follows:

- Block: The time stamp for the last change to a global data block is always the same as the time stamp for the interface and the data.
- Interface and data: The interface time stamps and the data are updated each time the global data block is changed. Even if you manually undo a change, for example remove a tag, that is also a change which will update the time stamp. However, if you undo the change using the "Undo" function, the time stamps will be reset to the value they had before the undone change.
- Load-relevant: The time stamp for "Load-relevant" is updated each time a global data block is changed. These changes can affect the data or the interface.

Instance data block time stamps

The following time stamps are generated for instance data blocks:

- Block: Created on, Modified on
- Interface: Modified on
- Data: Modified on
- Load-relevant: Modified on

A time stamp conflict will be reported during compilation of an instance data block if the interface time stamps of the instance data block are not identical to those of the function block.

The time stamps for instance data blocks are updated as follows:

- Block: The time stamp for the last change to an instance data block is always the same as the time stamp for the interface and the data.
- Interface and data: The interface time stamps and the data are updated each time the instance data block is changed. Even if you manually undo a change, for example cancel the tag retain setting, that is also a change which will update the time stamps. However, if you undo the change using the "Undo" function, the time stamps will be reset to the value they had before the undone change.

- **Load-relevant:** The time stamp for "Load-relevant" is updated each time an instance data block is changed. These changes can affect the data or the interface.

PLC data type time stamps

The following time stamps are generated for PLC data types:

- **Block:** Created on, Modified on
- **Interface:** Modified on
- **Load-relevant:** Modified on

The time stamps for PLC data types are updated as follows:

- **Block:** The time stamp for the last change to a PLC data type is always the same as the interface time stamp.
- **Interface:** The interface time stamp is updated each time the PLC data type is changed. Even if you manually undo a change, for example delete the content of a PLC data type, that is also a change which will update the time stamp. However, if you undo the change using the "Undo" function, the time stamp will be reset to the value it had before the undone change.
- **Load-relevant:** The time stamp for "Load-relevant" is updated each time a PLC data type is changed.

See also

Basics of block properties
 Overview of block properties
 Displaying and editing block properties
 Basic information on compiling blocks

1.2.4 Displaying and editing block properties

The properties of the blocks are block and CPU-specific. Not all properties are therefore available for all blocks or in all CPU families. Properties that can only be displayed are write-protected.

Displaying and editing properties of a closed block

To display and edit the properties of a closed block, follow these steps:

1. Open the "Program blocks" folder in the project tree.
2. Right-click the block whose properties you want to display or edit.
3. Select the "Properties" command in the shortcut menu.

The properties dialog box of the block opens.

4. In the area navigation, click a group whose properties you want to display or edit.
5. Change the relevant property.
6. Confirm your entries with "OK".

Displaying and editing properties of an open block

To display or edit the properties of an open block, follow these steps:

1. Select the "Inspector window" check box in the "View" menu.

The Inspector window opens.

2. Click the "Properties" tab.

The properties of the block are shown in the "Properties" tab of the Inspector window.

3. In the area navigation, click a group whose properties you want to display or edit.
4. Change the relevant property.

Result

The properties of the block will be changed. The changes are not saved until the project is saved.

See also

Basics of block properties

Overview of block properties

Block time stamps

1.3 Managing blocks

1.3.1 Opening blocks

You can open both the blocks in the project (offline blocks) and also the blocks in the device (online blocks).

To open an offline block, you have the following options:

- Open offline block directly

You can open a block directly if the corresponding block folder is open in the project tree or in the overview window.

- Find and open offline block

You can search for blocks within a project, a device, the "Program blocks" folder and in the Program editor, and then open these blocks.

Note the following special features when opening online blocks:

- You cannot edit online blocks.
- It is possible that another user can carry out a loading process on the selected CPU through joint parallel working on a CPU. Consequently, it can happen that the online block that you have open is deleted by the loading process if the block only exists in the device. In this case, the online block is closed and a message is displayed in the Inspector window.

Open offline block directly

To open an offline block directly, you have the following options:

1. Open the folder with the block you wish to open in the project tree or in the overview window.
2. Double-click on the block you wish to open.

The block will open in the program editor.

Find and open offline block

To find an offline block and then open it, follow these steps:

1. You can start the search process for the following three types:
 - In the shortcut menu of the project, of a device or of the "Program block" folder, select the "Open block/PLC data type" command or press the <F7> key. The search area depends on the selected element.
 - Press the <F7> key in the open program editor.

The "Open Block/PLC Data Type" dialog box opens.

2. Enter the name, the namespace, the address or the type of block that you are looking for. You can also enter only parts of the name of the field.

The block list is filtered further with each letter entered. All blocks are displayed whose name contains the text entered. It makes no difference here in which position the text occurs. No distinction is made between upper and lower case. Blocks which begin with the entered text are displayed at the start of the list. The block list is closed if there is no block that matches the input. You can show the complete block list at any time by clicking the button to the right of the text box. Keep in mind that there is no filtering in this case. If you want to filter for your inputs again, click the button once again.

3. In the block list, click the block you wish to open.

The block is opened in the program editor and displayed as selection in the project tree.

See also:

Overview of the general settings

Open online block

To open an online block directly, you have the following options:

1. Open the "Online access" folder in the project tree.
2. Click on the arrow symbol to the left of the interface to show all the objects that are arranged below the interface.
3. Double-click on the "Update accessible devices" command below the interface.

All devices that are accessible via this interface are displayed.

4. Open the folder of the device that contains the block you want to open.
5. Open the "Program blocks" folder.

All the device's blocks are displayed.

6. Double-click on the block you wish to open.

The block will open in the program editor.

See also

Saving blocks

Closing blocks

Renaming blocks

Deleting blocks offline

Deleting blocks online (S7-300, S7-400)

Opening know-how protected blocks

1.3.2 Saving blocks

Blocks are always saved together with the project. Faulty blocks can also be saved. This allows the error to be resolved at a convenient time.

Procedure

To save a block, follow these steps:

1. Select the "Save" or "Save as" command in the "Project" menu.

See also: Saving projects

See also

Opening blocks

Closing blocks

Renaming blocks

Deleting blocks offline

Deleting blocks online (S7-300, S7-400)

1.3.3 Closing blocks

Procedure

To close a block, follow these steps:

1. Click the "Close" button in the title bar of the program editor.

Note

Note that the block will not be saved on closing.

See also

Opening blocks

Saving blocks

Renaming blocks

Deleting blocks offline

Deleting blocks online (S7-300, S7-400)

1.3.4 Renaming blocks

Requirement

The "Program blocks" folder is opened in the project tree.

Procedure

To change the name of a block, follow these steps:

1. Right-click the block that you want to rename.
2. Select the "Rename" command in the shortcut menu.
The block name in the project tree changes to an input field.
3. Input the new name for the block.
4. Confirm your entry with the Enter key.

Result

The name of the block is now changed at all points of use in the program.

See also

Opening blocks

Saving blocks

Closing blocks

Deleting blocks offline

Deleting blocks online (S7-300, S7-400)

1.3.5 Deleting blocks offline

Requirement

The "Program blocks" folder opens in the project tree.

Procedure

To delete a block that exists offline, proceed as follows:

1. In the project tree in the "Program blocks" folder, right-click on the block that you want to delete.

2. Select the "Delete" command in the shortcut menu.
3. Confirm the safety prompt with "Yes".

The block is deleted offline from the project.

Note

If you are deleting organization blocks, note that events may be assigned to these blocks. If you delete such organization block the program cannot respond to parameterized events.

See also

Opening blocks

Saving blocks

Closing blocks

Renaming blocks

Deleting blocks online (S7-300, S7-400)

Downloading blocks for S7-1200/1500 (S7-1200, S7-1500)

1.3.6 Deleting blocks online (S7-300, S7-400)

Requirement

The "Program blocks" folder in the project tree is open.

Procedure

To delete a block that exists online, follow these steps:

1. In the "Program blocks" folder in the project tree, right-click on the block that you wish to delete from the device.
2. Select the "Delete" command from the shortcut menu.

The "Delete" dialog opens.

3. Select the "Delete from device" check box.
4. Click "Yes".

The block will be deleted from the device online.

See also

Opening blocks

Saving blocks

Closing blocks

Renaming blocks

Deleting blocks offline

1.3.7 Deleting CPU data blocks

You may delete CPU data blocks both in offline and online mode.

Deleting CPU data blocks in offline mode

Proceed as follows to delete a CPU data block from the offline project:

1. Right-click the CPU data blocks that you want to delete in the project navigation, "Program blocks" folder.
2. Select the "Delete" command from the shortcut menu.
3. Confirm the safety prompt with "Yes".

The CPU data block is deleted from the offline project.

Deleting CPU data blocks in online mode

Proceed as follows to delete a CPU data block from the online project:

1. Establish an online connection to the device containing the CPU data block that you want to delete.
2. Right-click the CPU data block that you want to delete from the device in the project navigation, "Program blocks" folder.
3. Select the "Delete" command from the shortcut menu.
The "Delete" dialog opens.
4. Select the "Delete from device" check box.
5. Click "Yes".

The CPU data block is deleted from the online device.

See also

CPU data blocks

1.4 Using external source files for STL and SCL

1.4.1 Basics of using external source files

Function

The textual programming languages STL and SCL allow you to enter the program code in any ASCII editor and save it as an external source file. This enables you to perform a range of tasks, for example:

- Declaring tags
- Specify block properties
- Programming blocks

In addition, you can also save PLC data types to external source files.

Note

Please note the following:

- For blocks to be generated from the external source file or PLC data types in the TIA Portal, the source file must be saved with one of the following character encodings:
 - ANSI
 - UTF-8 with BOM (Byte Order Mark)
 - UTF-16 with BOM, little and big endian format
 - UTF-32 with BOM, little and big endian format
 - UCS2 in little and big endian format
- Within software units you can use external source files only for optimized blocks in the SCL programming language and for PLC data types.
- Using external sources, blocks and PLC data types can only ever be generated in the respective environment, which means within a Software unit or directly under the PLC. This also means, for example, that no blocks or PLC data types are overwritten in other software units. If this was necessary due to the selected external source, the generation is not carried out.

You can import these source files to your project and use them to generate blocks or PLC data types. This enables you can generate several objects from one source file. Observe the following special features when generating blocks from a source file:

- If a block or a PLC data type with the same name already exists in the project, the block or PLC data type is overwritten in the project.
- If a block was programmed with its absolute block number instead of a symbolic name in the source file and this number is already assigned by a block in the project, the new generated block is initially assigned the next free symbolic name.
- If you have not explicitly defined the access mode for a block in the external source file, the block access mode is set depending on the CPU series used:
 - Blocks generated for a CPU of the S7-1200/1500 series are assigned "optimized" access mode by default.
 - Blocks generated for a CPU of the S7-300/400 product range are assigned "standard" access mode by default.

Organization blocks are the exception in this case, as they are always assigned the "standard" access mode by default, regardless of the CPU series. You have the option of changing the block access mode manually.

- It is possible that not all comments from the source file will be applied in the block.
- If you use absolute addressing in the external source file, a symbolic tag is created for each absolute address during the generation of the block. The names of these tags are made up of "Tag_" and a time stamp. This may result in relatively long tag names, which you can change manually if required.
- If you are using instructions in a different version in the external source file than in the target device, this may result in compilation errors. Correct the respective instructions in this case and start the compilation process once again. You can also select a different version for the target device.
- CPUs of the S7-300/400 series accept organization blocks with any name during import. If you import an OB with a standard name (for example, CYCL_EXC or CYC_INT5), the correct OB class is automatically assigned. If you import an OB with an unknown name, an OB 0 is created. You can change this number after the import and hereby assign the desired OB class. To change the OB number, select "Properties" in the shortcut menu.
- No automatic checking of the source is made when an SCL source is imported. As a result, blocks generated from this may contain errors which are only displayed at the next compilation.

You also have the option of saving existing blocks as external source files. You can either write only the program code of the selected block to a source file or additionally the program code of the blocks and PLC data types dependent on the selected block. When saving several blocks to a source file, all blocks must use the same programming language and none of the blocks may have know-how protection. You can also copy PLC data types individually as text and paste them into a text file.

See also

Rules for programming external source files
 Saving blocks as external source files
 Inserting external source files
 Opening and editing external source files
 Generating blocks from external source files

1.4.2 Rules for programming external source files

An external source file basically consists of continuous text. To compile the source into blocks, certain structures and syntax rules must however be adhered to.

Syntax rules

The syntax of the instructions in external source files is very similar to that in the creation of user programs in the program editor with STL or SCL. Note, however, the following additional syntax rules:

- Block call

When calling a block, transfer the parameters in the defined order in the ASCII editor. If you do not, the comment assignments for these lines may not match.

Enter the parameters in brackets. The individual parameters are separated by a comma.

- Upper or lower case

The program editor generally disregards upper or lower case. Jump labels are an exception to this. Character string entries are also case-sensitive ("STRING" data type). Keywords are displayed in upper case. For compilation purposes, however, case is disregarded; you can therefore specify keywords in upper or lower case or a mixture of the two.

- Semicolon

Mark the end of every instruction and every tag declaration with a semicolon. You can enter several instructions per line.

- Forward slashes

Begin every comment with two forward slashes (//) and end the comment with the <Enter> key.

- Use of String constants

To avoid compilation errors when using String constants, enter the text in the language of the target project. You can explicitly use the data type WSTRING for CPUs of the S7-1200/1500 series by using the prefix "WString#":

```
Operand := WString# '<String constant>';
```

See also

Basics of using external source files

Saving blocks as external source files

Inserting external source files

Opening and editing external source files

Generating blocks from external source files

1.4.3 Saving blocks as external source files

You have the following options for saving STL and SCL blocks and PLC data types as external source files:

- Copying a block as text

Note that with protected blocks only the information that you can also see without a password is copied to the source file.

- Generating external source file from blocks

You can either write only the program code of the selected block to a source file or additionally the program code of all the blocks and PLC data types dependent on the

selected block. All blocks must use the same programming language and none of the blocks may have know-how protection.

Note

When using the tabular block interface, note that not all initial values are exported to the source, e.g. for PLC data types.

Note that comments from ARRAY elements are not exported to the external source file.

Copying a block as text

To copy a block or PLC data type as text and save it to an external source file, follow these steps:

1. In the project tree, right-click on the block that you want to save in an external source file.
2. Select the "Copy as text" command in the shortcut menu.
3. Open an external text editor.
4. Paste the copied text from the clipboard.
5. Save the file with one of the following file name extensions:
 - ".scl", if you want to generate a source file for an SCL block
 - ".stl", if you want to generate a source file for an STL block
 - ".DB" if you wish to generate a source file for a data block
 - ".UDT", if you want to generate a source file for a PLC data type

Generating external source file from blocks

To generate external source files from STL or SCL blocks or PLC data types, follow these steps:

1. In the project tree or in the Overview window, select the blocks or the PLC data types from which you want to generate an external source file.

Note

Select only blocks with the same programming language and without know-how protection.

2. Select the "Generate source from blocks > Selected blocks only" command from the shortcut menu to save only the selected blocks or PLC data types to an external source file. Or select the command "Generate source from blocks > Including dependent blocks" if you also want to save the program code of the dependent blocks and referenced PLC data types to the external source file.

The "Save as" dialog opens.

3. Specify a path and a name for the external source file.

4. Click "OK".

You can also generate an opened STL or SCL block as external source file. To do this, follow these steps:

1. In the program editor, select the "Generate source from this block" command in the toolbar in the drop-down list "Generate source from block" to save only the open block in an external source file. Or select the command "Generate source from this block and all dependent blocks" if you also want to save the program code of the dependent blocks and referenced PLC data types to the external source file.

The "Save as" dialog opens.

2. Specify a path and a name for the external source file.
3. Click "OK".

Result

Depending on the selected option, either only the block or the block together with the dependent blocks and PLC data types are saved as an external source file. You can include this source file in a project in the TIA Portal and use it to generate other blocks. However, please note that you can use STL source files only in S7-300/400/1500 CPUs.

See also

Basics of using external source files
Rules for programming external source files
Inserting external source files
Opening and editing external source files
Generating blocks from external source files

1.4.4 Inserting external source files

Requirement

- An external source file is available and complies with the syntax and structure rules.
- The "External source files" folder is open in the project tree.

Procedure

Follow these steps to insert an external source file:

1. Double-click on the "Add new external file" command.
The "Open" dialog box opens.
2. Navigate to and select existing external source files.
3. Confirm your selection with "Open".

If there is already an external source file with this name in the project, the "Add external file" dialog opens. In this case, continue with step 4.

4. Select whether the new external source is to be renamed and inserted or whether the existing external source to be replaced.
5. Click "OK".

Result

The new source file will be added to the "External source files" folder.

See also

Basics of using external source files
Rules for programming external source files
Saving blocks as external source files
Opening and editing external source files
Generating blocks from external source files

1.4.5 Opening and editing external source files

By linking the files with the file name extensions ".stl", ".scl", ".db" and ".udt" to an editor you will be able to open and edit external source files with these formats directly. Use Notepad as editor because other text editors may not let you open several sources at the same time.

This means you do not need to insert the external source files again after editing.

Link files with the file name extensions ".stl", ".scl", ".db" and ".udt" to an editor

Proceed as follows to link files with the file name extensions ".stl", ".scl", ".db" and ".udt" to an editor:

1. Open Windows Explorer.
2. Right-click on an STL file.
3. Select "Properties" in the shortcut menu.
The "Properties" dialog box opens.
4. Click "Change" in the "File type" area on the "General" tab.
The "Open with" dialog box opens.
5. Select the text editor you want to link to the ".stl" file type.
6. Confirm your selection with "OK".
7. Close the "Properties" dialog with "OK".
8. Repeat steps 2 to 7 with the other file types in each case.

Opening and editing an external source file

To open an external source file, follow these steps:

1. Open the "External source files" folder in the project tree.

2. Double-click on the external source file you want to open.

The external source file will open in the linked editor and can be edited.

See also

Basics of using external source files

Rules for programming external source files

Saving blocks as external source files

Inserting external source files

Generating blocks from external source files

1.4.6 Generating blocks from external source files

Requirement

- The "External source files" folder is open in the project tree.
- An external source file is available.
- The external source file was saved with a permitted character encoding.

Procedure

To generate blocks from an external source file, follow these steps:

1. Open the external source file from which you wish to generate blocks.
2. Select the "Generate blocks from source" command in the "Edit" menu.
3. A prompt will appear telling you any existing blocks will be overwritten.
4. Confirm the safety prompt with "Yes".

Result

The external source file blocks will be generated and inserted in the "Program blocks" folder in the project tree. In the event of errors, information about the errors which have occurred will be displayed in the inspector window. This information, however, relates to the external source file and not to the block generated.

See also

Basics of using external source files

Rules for programming external source files

Saving blocks as external source files

Inserting external source files

Opening and editing external source files

Specifying block properties

1.5 Exporting and importing blocks in SIMATIC SD format (S7-1200, S7-1500)

1.5.1 Exporting and importing blocks in SIMATIC SD format (S7-1200, S7-1500)

Function

You can convert blocks of the LAD programming language, data blocks and PLC data types into text form by exporting them in SIMATIC SD format. You can use the created files independent of the TIA Portal. The files in text form can be edited with any text editor and managed in a version control system. This function is available for CPUs of the S7-1200 and S7-1500 series.

A SIMATIC SD export generates two different file formats. Files in s7dcl format contain the program code. The language-dependent comments are stored in the s7res files. If you export several blocks, one text file for the code and one text file for the comments is generated for each block. Blocks from software units can also be exported.

The program code is compiled in an easily readable format. The formats can be merged. There are further advantages as well:

- Compatibility and versionability
- No losses in user-generated files during the export and import of the files
- Support of integration in common version control systems such as Git
- Use for automation of engineering workflows
- Exportability of uncompileable blocks

Each block from TIA Portal is completely mapped via the associated s7dcl file. Export in SIMATIC SD format assigns the block according to the name contained in the text file. The block number is not included in the s7dcl file, since the number is automatically assigned when the text file is imported from TIA Portal. The text files in SIMATIC SD format begin with a pragma that contains general block attributes such as the type of block access or the author. Similar to the structure of the textual block interface, the s7dcl files contain a block declaration and declaration subsections. Data blocks and PLC data types are written completely with these sections. Functions and function blocks in LAD contain the networks with program code in addition. The s7dcl files therefore contain all information on block properties, variable declarations and, if necessary, program code in text format.

The s7res file is used to export the user texts of the selected block in all project languages. By default, the s7res file contains the languages that are set in the project navigation under Languages & Resources. A comment ID is assigned to each text element. A separate s7res file with the associated texts is generated for each exported block. The s7res file can be opened and edited with conventional text editors.

Note

Restrictions

You should note the following:

- If a block or a PLC data type with the same name already exists in the project, the block or PLC data type is overwritten in the project.
- This function is only supported for pure LAD blocks. It is not possible to export blocks with mixed programming languages.
- Know-how-protected blocks are excluded from import/export in SIMATIC SD format.

Procedure with TIA Portal Openness

You can export programming blocks of the LAD programming language, data blocks and PLC data types using TIA Portal Openness. Via this standardized method, you access the blocks of your TIA Portal project from a development environment. You can read the program code directly from TIA Portal by using files in SIMATIC SD format. The following requirements must be met in order for you to use this function:

- Installation of TIA Portal with TIA Portal Openness
- Microsoft Visual Studio with .Net Framework SDK in the version required for TIA Portal Openness
- Advanced programming knowledge in the specified tools as well as in C# / VB.net

You can find more information in the TIA Portal Openness documentation. You will find instructions for creating your program for access to blocks in the development environment under "Functions for accessing the data of a PLC device" in the "TIA Portal Openness API" section.

This excerpt from a programming code example in the development environment shows how an "Example_Block" LAD block is accessed using C#:

```
sw.BlockGroup.Blocks.Find("Example_Block").ExportAsDocuments(new
DirectoryInfo(@D:\Project\Exported_Files"), "Example_Block")
```

In detail:

```
sw.BlockGroup.Blocks.Find("Example_Block")
```

You have previously defined access to the TIA Portal using the expression `sw`. With `BlockGroup.Blocks.Find`, you access the desired block with the name "Example_Block".

```
ExportAsDocuments(new DirectoryInfo(@D:\Project\Exported_Files"),
"Example_Block")
```

With "ExportAsDocuments", you specify that the export is made SIMATIC SD format. You also provide the path to save the newly created text file and "Example_Block" as file name. The name of the exported block and the name of the file to be generated can differ. If the name of the function block contains characters that are not supported by the file system, you must select a different file name.

1.5.2 Representation of LAD program code in SIMATIC SD format (S7-1200, S7-1500)

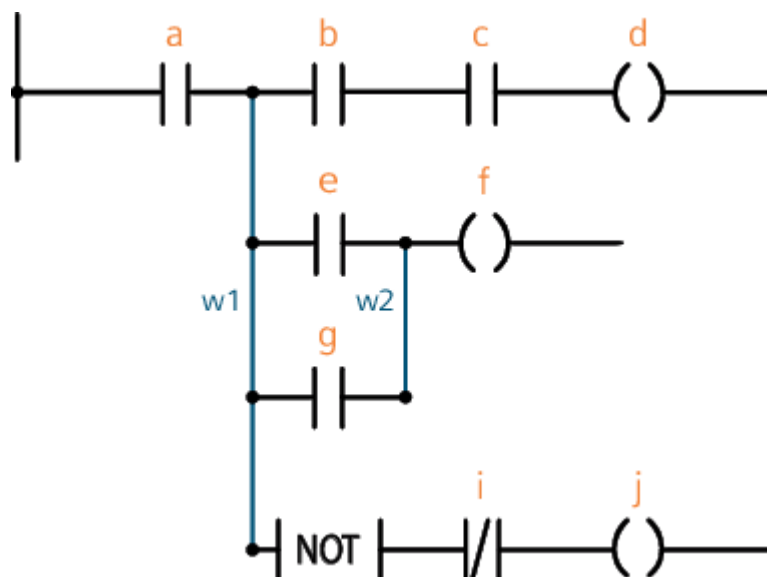
Compile mechanism for an export in SIMATIC SD format

The textual representation of LAD blocks in SIMATIC SD format is based on SCL program code. In comparison to programming in SCL, there are two additional keywords that are decisive when compiling a graphic block in the SIMATIC SD format. These keywords are "RUNG", "END_RUNG".

At least one rung branches off from the busbar of a network. A single rung can be linked to several parallel rungs via a branch. Each rung is identified by the keyword RUNG. If a RUNG starts at the busbar, it is given the designation RUNG wire#powerrail. If a RUNG starts at a branch, the branch must be given an identifier. When exporting in SIMATIC SD format, the identifiers are generated automatically. Its designation can consist of a combination of letters, symbols and numbers. By default, the branches are named in the order in which they occur according to the scheme w1, w2, w3 etc. The designation RUNG wire#w1 means that the RUNG starts at the branch with the identifier w1. To finish a RUNG, use the END_RUNG syntax. If a RUNG ends by closing a branch, the syntax is END_RUNG wire#1. Contacts and coils are arranged on a rung from left to right. In the textual representation, they appear in a list within a RUNG from top to bottom.

Principles for the structure of importable LAD programs:

- The number of rungs corresponds to the number of RUNGs.
- Contacts and coils of a rung appear as a list within a RUNG.
- Boxes are listed in the RUNG that is linked to the EN or IN parameter.
- Each branch is named with an identifier (w1, w2, w3, etc.).
- The branches are numbered according to the order in which they occur.



In this example, you can see 4 rungs and 2 branches. The first rung starts at the busbar and is designated as powerrail when exported in SIMATIC SD format. The following 3 rungs start with the first branch, so these RUNGs are given the identifier w1. The third rung ends with the second branch, which is given the identifier w2.



```
RUNG wire#powerrail
```

```

    Contact( a )

    WIRE w1

    Contact( b )

    Contact( c )

    Coil( d )

END_RUNG

RUNG wire#w1

    Contact( e )

    WIRE w2

    Coil( f )

END_RUNG

RUNG wire#w1

    Contact( g )

END_RUNG wire#w2

RUNG wire#w1

    Not

    I_Contact( i )

    Coil( j )

END_RUNG

```

Each LAD block can be exported in SIMATIC SD format. Bad blocks that cannot be compiled can also be exported. You can generate the text files at any time and save them in a version control system.

General structure

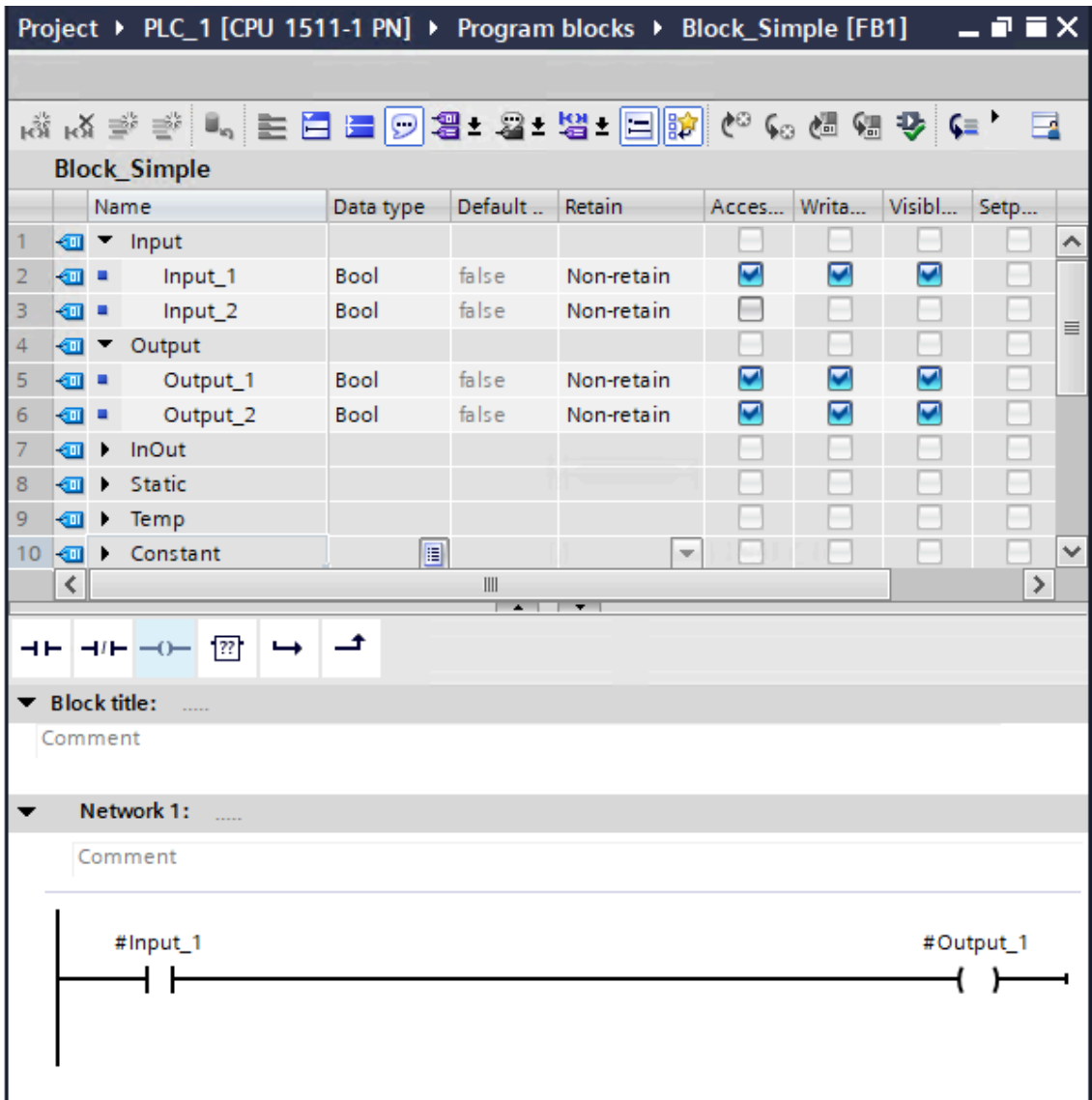
The text file follows the structure of the program block. Pragmas that refer to the general attributes of the block appear first in curly brackets.

The following FUNCTION_BLOCK section contains the block name describes the contents of the block, starting with the block interface. You can find the structure and rules of the text description of the block interface here. The program code of the individual networks follows underneath.

The block is addressed purely via the name, which is specified in the code after the keyword FUNCTION_BLOCK. The number of the block does not appear in the text file.

Example

Here you can see what a LAD block from the TIA Portal looks like after export in SIMATIC SD format.



When you export this block from the TIA Portal in SIMATIC SD format, the following text representation results:



```
{  
    S7_Optimize_Access := "TRUE" ;  
    S7_Language := "LAD";  
}  
  
FUNCTION_BLOCK "Block_Simple"  
    VAR_INPUT  
        Input_1 : Bool;  
    {  
        S7_Access := "ReadOnly := External";  
        S7_Visibility := "Hidden := External"  
    }  
    Input_2 : Bool;  
    END_VAR  
    VAR_OUTPUT  
        Output_1 : Bool;  
        Output_2 : Bool;  
    END_VAR  
    { S7_Language := "LAD" }  
  
    NETWORK  
        RUNG wire#powerrail  
            Contact( #Input_1 )  
            Coil( #Output_1 )  
        END_RUNG  
    END_NETWORK  
  
END_FUNCTION_BLOCK
```